Fall 2024 *Qatar University*

Project for CMPS 497/CMPE 471 Deep Learning:

Cross-prompt Automated Essay Scoring using Neural Models

Students:

Iheb Zouari	*202008203*
Mohamad Sandouk	*202007909*
Rufus	*202107326*
Mohamed Kasif	*202110389*

7 December 2024

Introduction

The objective of this project is to apply what we have learned in this course to tackle an example of a real-world problem, which is Automated Essay Scoring (AES), by designing, training, and evaluating several deep learning models for it. We will follow some of the work described in the paper titled “Conundrums in Cross-Prompt Automated Essay Scoring: Making Sense of the State of the Art” that was recently published at ACL 2024, the top conference in the Natural Language Processing (NLP) field.

1. Models and Training

A. Data Splitting

Approach A

For training, we generated 8 different sets from the ASAP dataset based on the `prompt_id` using the `splitData` function. In each set, The data rows that have different `prompt_id` as the one in the parameter of the function were used for training the model. On the other hand, the data rows that have the same `prompt_id` were put as test data, in a leave-one-prompt-out fashion. After that, we used each set to fine tune hyperparameters of each prompt model, using 7-fold cross validation.

The 86 features were then selected from the dataset as all columns starting at `mean_word` onward. While the label was the 'holistic' score column. That resulted in having the features (`X_trainData` and `X_testData`) and labels (`Y_trainData` and `Y_testData`) for each model set.

Approach B

Procedure was done similar to Approach A.

Approach C

For training, we used 7^{th} fold cross validation, wherein prompts 2 to 8 were used for training and one was used for validating, so at any time six prompts were training one validating and the target prompt was id = 1. The 86 features after the column narrativity were extracted from the dataset.xlsx file along with essay text and holistic score.

B. Models Training for the test sets

Approach A

The model was designed using a fully connected feed-forward neural network with (Dh) hidden layers and (D_k) hidden units with ReLU activation function. The output layer had a sigmoid activation function, because the output must be between 0 and 1. To conduct hyperparameter optimization using 7-fold cross-validation, we changed parameters such as number of hidden layers, learning rate, number of hidden units, and batch size. During training, we used the Quadratic Weighted Kappa (QWK) score to as an evaluation metric. Early stopping was applied to prevent overfitting. Moreover, the model was trained using the AdamW optimizer with mean squared error (MSE) as the loss function. For each epoch, the model was trained and validated, and if the validation score did not improve for four consecutive epochs (patience), the training was halted. After finding the optimal hyperparameters, the model was then retrained on the training set and evaluated using a test (unseen) and the final results were reported using the QWK score. And the trained model was then saved using TorchScript for future use.

Approach B

The models used for the test sets were trained using a two-step hyperparameter search strategy with 7-fold cross-validation. The model architecture consisted of a fully connected feed-forward neural network with (D_i) input features, (D_o) output units, and varying numbers of hidden layers and units, chosen from hyperparameter combinations. During training, the AdamW optimizer was used along with the mean squared error (MSE) as the loss function. The Quadratic Weighted Kappa (QWK) score was used as the evaluation metric for validation. Early stopping was implemented with a patience of 3 epochs to prevent overfitting. For each hyperparameter combination, the training process was repeated across all folds, and the average validation QWK was computed to identify the best hyperparameters. After selecting the optimal configuration, the model was retrained on the complete training dataset and evaluated using the test set. The final test QWK score was reported as the overall performance measure. The trained model was saved using TorchScript to ensure its deployment readiness.

Approach C

To evaluate the model's performance, it was run on a different test dataset, which should ideally not have been viewed by the model during training or validation. This stage is essential since it gives the model an objective assessment, here our target prompt was ID =1.
To measure how closely the model's predictions match the actual results, the QWK score was computed using the test set's predictions. We separated our dataset into several training and validation sets using a K-fold cross-validation technique with seven splits. This approach made sure that our model was tested on several data subsets, which enabled us to thoroughly validate its performance. In the training phase, we used the Adam optimizer and the Mean Squared Error loss function to train our model for each fold produced by the K-Fold cross-validation. Throughout the training phase, we established a learning rate schedule that progressively improved our model's predictions by decreasing linearly from an initial rate of 0.005 to zero. We

assessed the model on the validation set following each training epoch. The loss and the Quadratic Weighted Kappa (QWK) score, which evaluates the degree of agreement between the expected and actual scores, were computed in this case.

C. Models training for deployment

Approach A

Each one of the 8 models is trained on a specific subset of the data with the best hyperparameters set. Therefore, the models can generalize better when using them in a Voting Ensemble model. Where each model predicts the holistic score based on input features and then all the predictions are averaged to obtain the best possible prediction. The model was not trained again on all dataset because we wanted the 8 models to be biased to the training data it faced, making each one of them unique.

This ensemble voting method is deployed in a user-friendly based AES system interface, where the user provides a dataset containing essays to score with: essay_id, prompt_id, and 86 essay features starting with ‘mean_word’, so the deployed models will predict the holistic score of the provided essays based on the input features and will scale the score up based on the prompt_id.

Approach B

The model for deployment was trained following a similar two-step hyperparameter optimization process as outlined for the test sets. Initially, the best hyperparameters were identified using 7-fold cross-validation, and training was performed on a complete training dataset with early stopping applied to avoid overfitting. This resulted in a trained model with optimal hyperparameters. The second step focused on fine-tuning the batch size through another hyperparameter search. This process aimed to identify the batch size that maximized model performance. Once the final model configuration was determined, it was retrained on the full training dataset and tested on an unseen test set to confirm its performance. The final model, now fine-tuned and ready for deployment, was scripted using TorchScript and saved for integration into the production system. The deployed model can score new essays by processing input features provided through an interface, outputting a holistic score based on the learned relationships from the training data.

Approach C

For the deployment phase of our project, we initially used a methodical approach by employing K-fold cross-validation with seven splits. This method allowed us to regularly divide our dataset into distinct training and validation halves. This would increase the validity of our model's performance validation by ensuring that it was properly tested using a variety of data subsets. For each fold produced by the cross-validation, we actively trained our model using the Adam optimizer and the Mean Squared Error loss function. We meticulously planned the learning rate to decrease linearly from an initial rate of 0.005 to zero as training progressed. It was simpler to modify the model's parameters to progressively lower inaccuracy because of this gradual decline.

D. Loss Function Design

Approach A

Because our model solves a regression problem, we chose to use Mean Squared Error (MSE) loss function. Our decision was due to the fact that MSE is suitable for regression tasks because of its ability to calculate the average squared difference between the predicted and actual values, penalizing larger errors more heavily. Moreover, what makes MSE even more suitable for our model, is that we assumed that the variance of the scores is constant (homoscedastic).

Approach B

We designed a custom loss function, maskedLoss, using Mean Squared Error (MSE) to handle regression tasks effectively by calculating the average squared difference between true and predicted values. The core innovation is applying a mask to both y_true and y_pred, using prompt_ids to selectively ignore irrelevant traits. This ensures the loss focuses only on relevant data, allowing the model to learn important patterns without being influenced by noise. This approach is suitable as it retains MSE's benefits for regression while guiding the model to prioritize meaningful features.

Approach C

We have chosen to use the Mean Squared Error (MSE) loss function for creating the loss function for our project. We choose MSE for our regression model that predicts continuous output scores because it accurately represents the average squared difference between the estimated and real values.

E. Score ranges handling

Approach A

To handle varying score ranges across prompts and evaluation criteria, we scaled down scores to a standardized range and then scaled them back up to the original range. The ‘scaleDownTrait’ function was used to normalize the ‘holistic’ score to a common range [0,1] based on predefined ranges for each prompt, found in the SCORE_RANGES dictionary. Then, ‘holistic score was scalled up for validation with QWK and the scaleUpTrait function was used to revert the scores to their original range before testing with QWK in the test phase.

Approach B

We implemented a two-step scaling process to address the issue of different score ranges across various prompts and evaluation criteria. First, we normalized all scores to a standard [0, 1] range using the scaleDownTrait function. This function utilized predefined score ranges for each prompt, stored in the SCORE_RANGES dictionary, to adjust scores for specific traits like 'holistic'. This normalization step facilitated more effective model training and allowed for consistent application of evaluation metrics such as QWK (Quadratic Weighted Kappa). Following the model's processing, we employed the scaleUpTraits function to reverse the normalization, returning the scores to their original ranges. This step was crucial for accurately assessing the model's performance during the testing phase, particularly when applying the QWK metric to compare predictions against the original score scales.

Approach C

We used several crucial techniques when managing the score ranges for our project to make sure that our model could handle and anticipate the wide range of scores related to the essays.
Standardization: To increase learning effectiveness and stabilize the training process, we standardized the results. Usually, we used z-scores for normalization or scaled holistic ratings between 0 and 1. This assisted in resolving problems with different score magnitudes that may impact the performance of neural networks. Adapting the Model to the Score Range:
Output Layer Design: The output layer of our BERT Regressor was set up according to whether scores were normalized or maintained within their initial range. A sigmoid activation function was employed to limit outputs for normalized scores to the 0–1 range.
Training Modifications and Error Metrics:
Sensitivity of the Loss Function: In order to achieve our objective of accurate score predictions, we used the Mean Squared Error (MSE) loss function because of its ability to highlight greater prediction errors.
Frequent Monitoring: Throughout training, we kept a close eye on forecasts to make sure they were within predicted score ranges, modifying our strategy as needed.

F. Parameter Initialization

Approach A

The weights of all nn.Linear layers were initialized with He initialization (also known as Kaiming initialization) through nn.init.kaiming_normal_. This procedure preserved the variance of activations as they propagate through the network. It helped in maintaining stable training and it reduced the risk of vanishing or exploding gradients. The initialization supports networks with ReLU activations. Therefore, it optimizes both forward and backward passes, which will enhance the overall performance. Additionally, the biases were set to zero using layer_in.bias.data.fill_(0.0).

Approach B

Procedure was done similar to Approach A.

Approach C

Strategy for Initialization: BERT Pretrained Weights: We started the BERTRegressor using the pretrained weights from the BERT model. In order to facilitate quicker and more efficient learning in our particular natural language processing tasks, this method makes use of BERT's sophisticated language representations acquired through intensive pretraining on huge text corpora.

Random Initialization for new Layers: PyTorch's default parameters were used to initialize the custom layers that were added to integrate new features, such as the linear layers that combined BERT's outputs with other features. To preserve the variance of activations and gradients across layers and encourage stable convergence, this usually entails the Xavier uniform initialization. Should we run into problems with convergence or model performance, we were ready to investigate more specific initialization techniques as He Initialization for layers with ReLU activations or Zero Initialization for biases.

2. Experimental Evaluation

A. Performance for each prompt

Approach A

Model	Step 1					Step 2					
	Best Hyperparameters				Score	Best Hyperparameters				QWK Score	
	HS	LR	HL	BS	QWK	HS	LR	HL	BS	Validation	Test
1	**32**	**0.001**	**4**	4	0.517893	32	0.001	4	**16**	**0.7765**	**0.6392**
2	**16**	**0.001**	**8**	4	0.630729	16	0.001	8	**16**	**0.7771**	**0.5760**
3	**32**	**0.001**	**4**	4	0.59423	32	0.001	4	**16**	**0.8037**	**0.6240**
4	**32**	**0.001**	**4**	4	0.590126	32	0.001	4	**4**	**0.7701**	**0.6195**
5	**32**	**0.001**	**4**	4	0.739745	32	0.001	4	**16**	**0.7764**	**0.7277**
6	**16**	**0.001**	**8**	4	0.534789	16	0.001	8	**16**	**0.7835**	**0.5134**
7	**32**	**0.001**	**4**	4	0.698642	32	0.001	4	**4**	**0.7775**	**0.7445**
8	**32**	**0.001**	**2**	4	0.57726	32	0.001	2	**32**	**0.7836**	**0.5819**

The test QWK scores given here are observed by comparing the true holistic scores of each prompt to the predicted scores by the best model at each step.

While the validation QWK scores given here are an average of the scores across the folds of the specified hyperparameter set.

We can see that the validation scores are higher than test scores due to the fact that the hyperparameters were tuned to achieve the highest validation score, while test score are obtained from data that is unseen for the model.

We can see that the highest validation score ‘0.8037’ was achieved with model 3, while the highest test score ‘0.7445’ is achieved by model 7.

Approach B

Model	Step 1					Step 2					
	Best Hyperparameters				Score	Best Hyperparameters				QWK Score	
	HS	LR	HL	BS	QWK	HS	LR	HL	BS	Validation	Test
1	**32**	**0.001**	**1**	4	0.8447	**32**	**0.001**	**1**	**16**	**0.7720**	**0.8466**
2	**32**	**0.001**	**2**	4	0.7211	**32**	**0.001**	**2**	**16**	**0.7918**	**0.7363**
3	**32**	**0.001**	**2**	4	0.7478	**32**	**0.001**	**2**	**16**	**0.7953**	**0.7253**
4	**32**	**0.001**	**2**	4	0.6132	**32**	**0.001**	**2**	**32**	**0.8096**	**0.6113**
5	**32**	**0.001**	**2**	4	0.6902	**32**	**0.001**	**2**	**16**	**0.7854**	**0.7080**
6	**32**	**0.001**	**1**	4	0.7101	**32**	**0.001**	**1**	**16**	**0.7835**	**0.7571**
7	**32**	**0.001**	**1**	4	0.8173	**32**	**0.001**	**1**	**8**	**0.7699**	**0.8296**
8	**32**	**0.001**	**1**	4	0.7726	**32**	**0.001**	**1**	**16**	**0.7811**	**0.7930**

From the data shown above, we see that the highest validation QWK score, 0.8096, is achieved by model 4. Meanwhile, the highest test QWK score, 0.8466, is attained by model 1.

a. Main observation

Main observation on the results was made by running a test on our AES system, using data (Essays to score.csv) from the same dataset of the training (dataset.csv)

Approach A

```
AESsystem()
```

```
    Welcome to the Deep-Learning-based AES system.
    This system predicts scores based on the chosen approach.
    Approach A: Predicts holistic scores.
    Approach B: Predicts holistic and trait scores.

Type A or B A
(type 'exit' to exit) Paste the csv file path:  Essays to score.csv
File successfully read...
CSV file meets the requirements...
Data saved to Scored Essays (Approach A).csv!
```

Approach B

```
AESsystem()
```

```
    Welcome to the Deep-Learning-based AES system.
    This system predicts scores based on the chosen approach.
    Approach A: Predicts holistic scores.
    Approach B: Predicts holistic and trait scores.

Type A or B B
(type 'exit' to exit) Paste the csv file path:  Essays to score.csv
File successfully read...
CSV file meets the requirements...
Data saved to Scored Essays (Approach B).csv!
```

Approach A

Essay to score passed by the user

```
pd.read_csv('Essays to score.csv')
```

	essay_id	prompt_id	mean_word	word_var	mean_sent	sent_var	ess_char_len
0	1	1	0.481388	0.486119	0.086992	0.075307	0.394701
1	2	1	0.455093	0.433420	0.086329	0.062532	0.487845
2	3	1	0.483844	0.444176	0.080310	0.081166	0.325321
3	4	1	0.741008	0.567131	0.077239	0.059663	0.686424
4	5	1	0.485833	0.430699	0.054213	0.019130	0.549577
...	...	...	...	...	...	...	...
12971	21626	8	0.363237	0.176943	0.112795	0.010498	0.855434
12972	21628	8	0.294699	0.146153	0.047816	0.008641	0.531157
12973	21629	8	0.516168	0.363221	0.065713	0.011941	0.889332
12974	21630	8	0.426468	0.378559	0.043018	0.002492	0.585494
12975	21633	8	0.442089	0.398349	0.050157	0.001436	0.490528

12976 rows × 88 columns

Scored Essays based on deployed model predictions

```
pd.read_csv("Scored Essays (Approach A).csv")
```

	essay_id	prompt_id	holistic	mean_word	word_var	mean_sent	sent_var	ess_char_len	word_count	prep_comma
0	1	1	8	0.481388	0.486119	0.086992	0.075307	0.394701	0.425614	0.522059
1	2	1	9	0.455093	0.433420	0.086329	0.062532	0.487845	0.531695	0.514706
2	3	1	8	0.483844	0.444176	0.080310	0.081166	0.325321	0.350582	0.301471
3	4	1	10	0.741008	0.567131	0.077239	0.059663	0.686424	0.667529	0.566176
4	5	1	9	0.485833	0.430699	0.054213	0.019130	0.549577	0.591203	0.411765
...	...	...	...	...	...	...	...	...	...	...
12971	21626	8	40	0.363237	0.176943	0.112795	0.010498	0.855434	0.987089	0.767045
12972	21628	8	37	0.294699	0.146153	0.047816	0.008641	0.531157	0.634977	0.403409
12973	21629	8	42	0.516168	0.363221	0.065713	0.011941	0.889332	0.953052	0.852273
12974	21630	8	40	0.426468	0.378559	0.043018	0.002492	0.585494	0.654930	0.613636
12975	21633	8	39	0.442089	0.398349	0.050157	0.001436	0.490528	0.544601	0.454545

12976 rows × 89 columns

True values of the Holistic scores

```
pd.read_csv('dataset.csv')
```

	essay_id	prompt_id	holistic	mean_word	word_var	mean_sent	sent_var
0	1	1	8	0.481388	0.486119	0.086992	0.075307
1	2	1	9	0.455093	0.433420	0.086329	0.062532
2	3	1	7	0.483844	0.444176	0.080310	0.081166
3	4	1	10	0.741008	0.567131	0.077239	0.059663
4	5	1	8	0.485833	0.430699	0.054213	0.019130
...	...	...	...	...	...	...	...
12971	21626	8	35	0.363237	0.176943	0.112795	0.010498
12972	21628	8	32	0.294699	0.146153	0.047816	0.008641
12973	21629	8	40	0.516168	0.363221	0.065713	0.011941
12974	21630	8	40	0.426468	0.378559	0.043018	0.002492
12975	21633	8	40	0.442089	0.398349	0.050157	0.001436

12976 rows × 89 columns

Based on the true labels from the original dataset, we can see that the model has a good performance, but this performance might get slightly worse when tested with external test data.

Approach B

Essay to score passed by the user

```
pd.read_csv('Essays to score.csv')
```

	essay_id	prompt_id	mean_word	word_var	mean_sent	sent_var	ess_char_len
0	1	1	0.481388	0.486119	0.086992	0.075307	0.394701
1	2	1	0.455093	0.433420	0.086329	0.062532	0.487845
2	3	1	0.483844	0.444176	0.080310	0.081166	0.325321
3	4	1	0.741008	0.567131	0.077239	0.059663	0.686424
4	5	1	0.485833	0.430699	0.054213	0.019130	0.549577
...	...	...	...	...	...	...	...
12971	21626	8	0.363237	0.176943	0.112795	0.010498	0.855434
12972	21628	8	0.294699	0.146153	0.047816	0.008641	0.531157
12973	21629	8	0.516168	0.363221	0.065713	0.011941	0.889332
12974	21630	8	0.426468	0.378559	0.043018	0.002492	0.585494
12975	21633	8	0.442089	0.398349	0.050157	0.001436	0.490528

12976 rows × 88 columns

Scored Essays based on deployed model B predictions

```
pd.read_csv("Scored Essays (Approach B).csv")
```

t_id	holistic	content	organization	word_choice	sentence_fluency	conventions	prompt_adherence
1	9	4	4	4	4	4	0
1	9	4	4	4	4	4	0
1	8	4	3	3	4	4	0
1	10	5	5	4	4	4	0
1	9	4	4	4	4	4	0
...	...	...	...	...	...	...	...
8	40	8	8	8	8	8	0
8	37	8	7	7	7	7	0
8	42	9	9	9	9	8	0
8	41	8	8	8	8	8	0
8	39	8	8	8	8	8	0

True values of the scores

```
pd.read_csv("dataset.csv")
```

say_text	holistic	content	organization	word_choice	sentence_fluency	conventions	prompt_adherenc
ear local spaper, I k effects ompute...	8	4	3.0	3.0	3.0	3.0	Na
"Dear @CAPS1 CAPS2, I ieve that g comp...	9	4	4.0	4.0	3.0	4.0	Na
"Dear, @CAPS1 @CAPS2 @CAPS3 lore and e peop...	7	3	3.0	3.0	4.0	4.0	Na
ear Local wspaper,)CAPS1 I ve found tha...	10	5	4.0	5.0	4.0	4.0	Na

Based on the true labels from the original dataset, we can see that the model has a good performance, but this performance might get slightly worse when tested with external test data.

b. Performance discrepancy across prompts

Approach A & Approach B

There is a discrepancy between model performance across different prompts and that is due to many reasons, but is mainly because of the nature of the data. First each prompt has a different number of essays available in the dataset and each prompt has a different score range and different scores distribution. Also, new models are created for each fold with different initializations causing some difference between the results.

B. Best hyperparameters for each prompt

HS: *number of hidden units* LR: *learning rate* HL: *number of hidden layers* BS: *Batch size*

Approach A

The best hyperparameter values when the batch size is fixed at 4

Prompt	Step 1				
	Best Hyperparameters				Score
	HS	LR	HL	BS	QWK
1	**32**	**0.001**	**4**	4	0.51789
2	**16**	**0.001**	**8**	4	0.63073
3	**32**	**0.001**	**4**	4	0.59423
4	**32**	**0.001**	**4**	4	0.59013
5	**32**	**0.001**	**4**	4	0.73974
6	**16**	**0.001**	**8**	4	0.53479
7	**32**	**0.001**	**4**	4	0.69864
8	**32**	**0.001**	**2**	4	0.57726

The best hyper parameters when batch size is also considered as a hyper parameter

Prompts	Step 2				
	Best Hyperparameters				Score
	HS	LR	HL	BS	QWK
1	32	0.001	4	**16**	**0.6392**
2	16	0.001	8	**16**	**0.576**
3	32	0.001	4	**16**	**0.624**
4	32	0.001	4	**4**	**0.6195**
5	32	0.001	4	**16**	**0.7277**
6	16	0.001	8	**16**	**0.5134**
7	32	0.001	4	**4**	**0.7445**
8	32	0.001	2	**32**	**0.5819**

Approach B

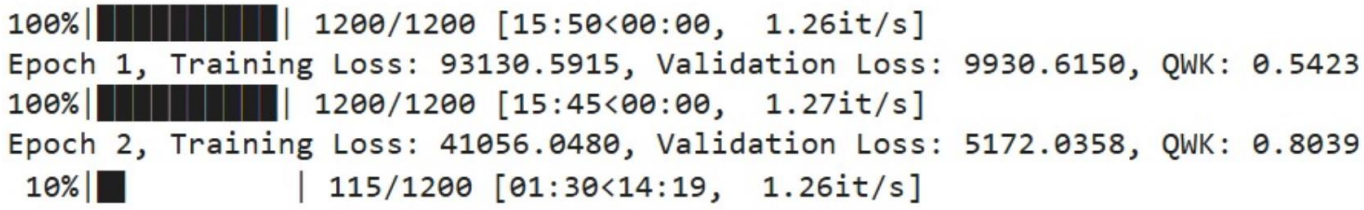
The best hyperparameter values when the batch size is fixed at 4

Model	Step 1				
	Best Hyperparameters				Score
	HS	LR	HL	BS	QWK
1	**32**	**0.001**	**1**	4	0.8447
2	**32**	**0.001**	**2**	4	0.7211
3	**32**	**0.001**	**2**	4	0.7478
4	**32**	**0.001**	**2**	4	0.6132
5	**32**	**0.001**	**2**	4	0.6902
6	**32**	**0.001**	**1**	4	0.7101
7	**32**	**0.001**	**1**	4	0.8173
8	**32**	**0.001**	**1**	4	0.7726

The best hyper parameters when batch size is also considered as a hyper parameter

Step 2					
Best Hyperparameters				QWK Score	
HS	LR	HL	BS	Validation	Test
32	**0.001**	**1**	**16**	**0.7720**	**0.8466**
32	**0.001**	**2**	**16**	**0.7918**	**0.7363**
32	**0.001**	**2**	**16**	**0.7953**	**0.7253**
32	**0.001**	**2**	**32**	**0.8096**	**0.6113**
32	**0.001**	**2**	**16**	**0.7854**	**0.7080**
32	**0.001**	**1**	**16**	**0.7835**	**0.7571**
32	**0.001**	**1**	**8**	**0.7699**	**0.8296**
32	**0.001**	**1**	**16**	**0.7811**	**0.7930**

Approach C



First Epoch:

The validation loss was much smaller at 9,930.6150 than the training loss, which was measured at 93,130.5915. With a score of 0.5423, the QWK (Quadratic Weighted Kappa) metric—which gauges the degree of agreement between forecasts and ground truth—showed moderate success thus far. These outcomes are indicative of the early stages of training, when the model has not yet reached convergence but is starting to identify patterns in the data.
Second Epoch: The model's ability to match the training data significantly improved, as evidenced by the training loss falling to 41,056.0480.
Likewise, the validation loss dropped to 5,172.0358, suggesting better generalization to data that hasn't been seen yet. With a significant improvement of 0.8039, the QWK score demonstrated increased alignment between the goal outputs and the projections.

Unfortunately, due to memory constraints our training process got interrupted.

Discussion on the best hyper-parameters.

Approach A

The QWK scores vary from 0.51789 to 0.73974 in step 1, reflecting variation in performance across prompts due to different feature inputs for different prompts giving rise to different models that predict the output with varying accuracy.

In step 2, batch size is varied between 4, 8, 16, 32 while the best of the other hyperparameters which were found in step 1 is fixed for step 2. Generally, this variation in batch size had an improving effect on the QWK scores for most prompts. The QWK score for prompt 1, for example, rose from 0.51789 to 0.63922, using a batch size of 16. However, we cannot clearly say that the best varying the batch sizes have improved the performance as there are also cases when the QWK scores dropped after varying the batch size like for prompt 2 when the QWK score fell from 0.63073 to 0.576.

Overall, step 2 highlights the selection of the correct batch size as an important issue for boosting QWK, observing that usually medium-sized batches like 16 make a good balance between explosive learning and vanishing learning, thus improving consistency in scores.

Approach B

The QWK scores in Step 1 range from 0.6132 to 0.8447 because the batch size was fixed and the other hyperparameters were changing. In Step 2, after changing the batch size we noticed shifts in QWK scores. Prompt 1 had the QWK score falling down from 0.8447 to 0.7720 when the batch size was increased from 4 to 16. Moreover, increasing the batch size from 4 to 32 in prompt 4 led to a drop in the QWK score from 0.6132 to 0.6113 On the other hand, the QWK score in prompt 7 improved from 0.8173 to 0.8296 when adjusting the batch size from 4 to 8. This shows that some batch size adjustments can enhance model performance.

These results show that changing the batch size can either improve or lower model performance. This highlights the importance of careful tuning to get the best results.

C. Batch size effect

Changing the batch size shows very consistent changes to the QWK scores. Small batch sizes, such as 4, allowed for more frequent updates that resulted in better generalization but at times made training noisier and slower. Setting the batch size to 8, 16, or 32 smoothed out gradient updates and made training more stable, improving scores. As an example, prompt 1's score increases from 0.51789 with batch size 4 to 0.63922 with batch size 16.

Larger batch sizes occasionally reduced scores due to the less frequent updates that resulted in smoother models. Medium-sized batches maybe got the best balance in most cases, again showing the importance of tuning batch size for consistent performance.

D. Comparison with state-of-the-art

Approach A

Our model:

Prompt	Step 1					Step 2				
	Best Hyperparameters				Score	Best Hyperparameters				Score
	HS	LR	HL	BS	QWK	HS	LR	HL	BS	QWK
1	**32**	**0.001**	**4**	4	0.51789	32	0.001	4	**16**	**0.6392**
2	**16**	**0.001**	**8**	4	0.63073	16	0.001	8	**16**	**0.576**
3	**32**	**0.001**	**4**	4	0.59423	32	0.001	4	**16**	**0.624**
4	**32**	**0.001**	**4**	4	0.59013	32	0.001	4	**4**	**0.6195**
5	**32**	**0.001**	**4**	4	0.73974	32	0.001	4	**16**	**0.7277**
6	**16**	**0.001**	**8**	4	0.53479	16	0.001	8	**16**	**0.5134**
7	**32**	**0.001**	**4**	4	0.69864	32	0.001	4	**4**	**0.7445**
8	**32**	**0.001**	**2**	4	0.57726	32	0.001	2	**32**	**0.5819**

State-of-the-art model:

	Model	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**Avg.**
1	Hi att (Dong et al., 2017)	.372	.465	.432	.523	.586	.574	.514	.323	.474
2	PAES (Ridley et al., 2020)	.746	.591	.608	.641	.727	.609	.707	.635	.658
3	PMAES (Chen and Li, 2023)	.758	**.674**	.658	.625	.735	.578	**.749**	**.718**	.687
4	Existing Feats	.744	.601	.657	.653	.778	.620	.704	.430	.648
5	All Feats	.735	.538	.602	.587	.722	.584	.689	.523	.623
6	Existing Feats Filtered	**.829**	.612	.621	.621	.767	.655	.739	.570	.677
7	All Feats Filtered	.820	.601	**.685**	**.666**	**.786**	**.682**	.693	.654	**.698**

We can observe the following when we compare the state-of-the-art models with our models

Most of our predictions are consistently close to the QWK of the best score among all the state-of-the-art models as shown by the table below which compares our model to state-of-the-art model's best QWK scores.

Prompt	Best QWK (State-of-art models)	Our model's QWK
1	0.829	0.6392
2	0.674	0.576
3	0.685	0.624
4	0.666	0.6195
5	0.786	0.7277
6	0.682	0.5134
7	0.749	0.7445
8	0.718	0.5819

Our model also outperformed one of the older models like Hi att on almost all prompts in terms of QWK scores. However, we can observe that the models trained on existing features, all features and all features filtered perform better on average than our model. Small discrepancies like this are expected as these models are trained on most probably a bigger dataset with more efficient programs and with better initialization. But our model performed relatively well considering the limited dataset and programming constraints that we had in our project.

Approach B

Our model:

Model¤	Step·1¤					Step·2¤					
	Best·Hyperparameters¤				Score¤	Best·Hyperparameters¤				QWK·Score¤	
	HS¤	LR¤	HL¤	BS¤	QWK¤	HS¤	LR¤	HL¤	BS¤	Validation¤	Test¤
1¤	**32**¤	**0.001**¤	**1**¤	4¤	0.8447¤	**32**¤	**0.001**¤	**1**¤	**16**¤	**0.7720**¤	**0.8466**¤
2¤	**32**¤	**0.001**¤	**2**¤	4¤	0.7211¤	**32**¤	**0.001**¤	**2**¤	**16**¤	**0.7918**¤	**0.7363**¤
3¤	**32**¤	**0.001**¤	**2**¤	4¤	0.7478¤	**32**¤	**0.001**¤	**2**¤	**16**¤	**0.7953**¤	**0.7253**¤
4¤	**32**¤	**0.001**¤	**2**¤	4¤	0.6132¤	**32**¤	**0.001**¤	**2**¤	**32**¤	**0.8096**¤	**0.6113**¤
5¤	**32**¤	**0.001**¤	**2**¤	4¤	0.6902¤	**32**¤	**0.001**¤	**2**¤	**16**¤	**0.7854**¤	**0.7080**¤
6¤	**32**¤	**0.001**¤	**1**¤	4¤	0.7101¤	**32**¤	**0.001**¤	**1**¤	**16**¤	**0.7835**¤	**0.7571**¤
7¤	**32**¤	**0.001**¤	**1**¤	4¤	0.8173¤	**32**¤	**0.001**¤	**1**¤	**8**¤	**0.7699**¤	**0.8296**¤
8¤	**32**¤	**0.001**¤	**1**¤	4¤	0.7726¤	**32**¤	**0.001**¤	**1**¤	**16**¤	**0.7811**¤	**0.7930**¤

The table below illustrates the difference between our model's QWK and the state-of-the-art model's QWK.

Prompt	Best QWK (State-of-art models)	Our model's QWK
1	0.829	0.8466
2	0.674	0.7363
3	0.685	0.7253
4	0.666	0.6113
5	0.786	0.7080
6	0.682	0.7571
7	0.749	0.8296
8	0.718	0.7930

As evident in the table above, our model performs better than the state-of-the-art models in 6 out of 8 prompts. This indicates that the predictive performance has improved significantly. Our model shows better QWK scores in Prompts 1, 2, 3, 6, 7, and 8. However, the SOTA models perform better in Prompts 4 and 5, where they score 0.666 and 0.786 compared to our 0.6113 and 0.7080. In this case, further optimization to our model might help close the performance gap.

3. Student Contributions

The table below presents student contributions in the project so far.

Table. Student Contributions in Approach A

Student	responsibilities and contributions	
Iheb Zouari	Models training AES system interface, report	25%
Mohamad Aljawad	Data acquisition/scaling and deployed model	25%
Rufus	Hyperparameter tuning,model training and report	25%
Mohamed Kasif	Setup, report, Loss function, README file	25%

Table. Student Contributions in Approach B

Student	responsibilities and contributions	
Iheb Zouari	Models training, Loss function, Handling of QWK scores, Data scaling	25%
Mohamad Aljawad	Data acquisition, deployed model, report	25%
Rufus	AES system, Hyperparameter tuning, model training and report	25%
Mohamed Kasif	Data splitting, model training, report, README file	25%

Table. Student Contributions in Approach C

Student	responsibilities and contributions	
Iheb Zouari	Loss function, Data scaling	25%
Mohamad Aljawad	Data acquisition, Pre-trained model, report	25%
Rufus	Data validation, debugging, model training, report	25%
Mohamed Kasif	Data splitting, data handling, model training, report, README file	25%